

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN 1



Đề tài: Phát triển hệ thống Microservices cho thương mại điện tử

Giảng viên: Kim Ngọc Bách

Nhóm lớp: 03

Nhóm bài tập: 18

Nguyễn Văn Thái - B22DCCN783

Nguyễn Tiến Đạt - B22DCCN196

Hà Nội – 2026

Lời cảm ơn

Nhóm chúng em xin gửi lời cảm ơn chân thành tới Quý Thầy/Cô tại Học viện Công nghệ Bưu chính Viễn thông nói chung và Khoa Công nghệ Thông tin nói riêng đã trang bị cho chúng em nền tảng kiến thức quan trọng về phát triển hệ thống thông tin, kiến trúc phần mềm và hệ thống phân tán.

Đặc biệt, nhóm xin bày tỏ lòng biết ơn tới Thầy Kim Ngọc Bách đã định hướng, hướng dẫn và tạo điều kiện để nhóm hoàn thành đề tài “Phát triển hệ thống Web bán gạo theo kiến trúc Microservices”. Quá trình thực hiện đề tài giúp nhóm hiểu rõ hơn cách tách nghiệp vụ thành các dịch vụ độc lập, sử dụng API Gateway, triển khai bằng Docker Compose, kết hợp cơ sở dữ liệu quan hệ và hàng đợi thông điệp để mô phỏng luồng thương mại điện tử.

Do giới hạn về thời gian và phạm vi bài tập lớn, báo cáo khó tránh khỏi thiếu sót. Nhóm rất mong nhận được góp ý của Thầy/Cô để tiếp tục hoàn thiện hệ thống và nâng cao chất lượng sản phẩm trong các lần phát triển tiếp theo.

Mục lục

Lời cảm ơn	2
Danh mục từ viết tắt	5
BÁO CÁO BÀI TẬP LỚN	6
Chương 1. TỔNG QUAN ĐỀ TÀI	6
1.1. Bối cảnh	6
1.2. Mục tiêu của đề tài	6
1.3. Phạm vi	7
Chương 2. CƠ SỞ LÝ THUYẾT VỀ HỆ THỐNG PHÂN TÁN	7
2.1. Kiến trúc Microservices	7
2.2. API Gateway	7
2.3. Giao tiếp REST và Message Broker	8
2.4. Database-per-service và polyglot persistence	8
2.5. Containerization và Docker Compose	8
2.6. Observability và xử lý lỗi	8
Chương 3. THIẾT KẾ HỆ THỐNG	9
3.1. Kiến trúc tổng quan	9
3.2. Danh sách microservice và trách nhiệm	9
3.3. Thiết kế API Gateway và SQL Adapter	10
3.4. Hai luồng dữ liệu trong hệ thống	11
3.4.1. Luồng storefront/admin mới	11
3.4.2. Luồng microservice gốc qua RabbitMQ	12
3.5. Thiết kế dữ liệu	13
3.6. Thiết kế frontend	13
Chương 4. CÀI ĐẶT VÀ TRIỂN KHAI	14
4.1. Cấu trúc dự án	14
4.2. Công nghệ sử dụng	15
4.3. Triển khai bằng Docker Compose	15
4.4. Hướng dẫn chạy và kiểm thử	16
Chương 5. KẾT QUẢ MINH HỌA	17
5.1. Kết quả chức năng	17

5.2. Demo	18
5.3. Kịch bản kiểm thử đề xuất	20
Chương 6. ĐÁNH GIÁ VÀ KẾT LUẬN	21
6.1. Những gì đã đạt được	21
6.2. Hạn chế	21
6.3. Hướng phát triển	22
6.4. Kết luận	22
TÀI LIỆU THAM KHẢO	22

Danh mục từ viết tắt

Từ viết tắt	Nghĩa đầy đủ	Ý nghĩa trong báo cáo
API	Application Programming Interface	Giao diện lập trình ứng dụng, nơi frontend hoặc service khác gửi request.
DB	Database	Cơ sở dữ liệu dùng để lưu thông tin nghiệp vụ.
HTTP	HyperText Transfer Protocol	Giao thức truyền thông chính của web và REST API.
REST	Representational State Transfer	Phong cách thiết kế API dựa trên tài nguyên và phương thức HTTP.
UI	User Interface	Giao diện người dùng.
SQL	Structured Query Language	Ngôn ngữ truy vấn cơ sở dữ liệu quan hệ.
AMQP	Advanced Message Queuing Protocol	Giao thức hàng đợi thông điệp, được RabbitMQ hỗ trợ.
JWT	JSON Web Token	Token dùng để biểu diễn thông tin đăng nhập/phân quyền.
CRUD	Create, Read, Update, Delete	Bốn thao tác cơ bản trên dữ liệu.
Docker	Container platform	Nền tảng đóng gói và chạy ứng dụng bằng container.

BÁO CÁO BÀI TẬP LỚN

Chương 1. TỔNG QUAN ĐỀ TÀI

1.1. Bối cảnh

Trong lĩnh vực thương mại điện tử, các hệ thống bán hàng trực tuyến thường phải xử lý nhiều nghiệp vụ khác nhau như quản lý sản phẩm, tài khoản người dùng, giỏ hàng, đơn hàng, tồn kho, thông báo và quản trị hệ thống. Nếu toàn bộ các chức năng này được đặt trong một ứng dụng nguyên khối, việc bảo trì và mở rộng có thể trở nên khó khăn khi hệ thống phát triển.

Đối với một website bán game bida, yêu cầu nghiệp vụ không chỉ dừng ở việc hiển thị danh sách sản phẩm. Hệ thống cần có trang chủ, danh mục sản phẩm, chi tiết sản phẩm, giỏ hàng, đặt hàng, tài khoản người dùng, đánh giá sản phẩm, bài viết, quản lý tồn kho và khu vực quản trị. Vì vậy, việc áp dụng kiến trúc microservices giúp chia nhỏ hệ thống theo từng miền nghiệp vụ, qua đó dễ phát triển, kiểm thử và triển khai hơn.

Dự án được sử dụng trong báo cáo là hệ thống “Billiard Shop Microservices”, một hệ thống bán game bida triển khai bằng Node.js, API Gateway, PostgreSQL, RabbitMQ, SQL Server và frontend tĩnh chạy qua Nginx. Điểm đặc biệt của dự án là kiến trúc lai: frontend/storefront mới sử dụng API Gateway với SQL Adapter để thao tác trực tiếp với SQL Server, đồng thời vẫn giữ lại luồng microservice gốc dùng PostgreSQL và RabbitMQ để minh họa mô hình phân tán.

1.2. Mục tiêu của đề tài

Mục tiêu chính của đề tài là xây dựng và trình bày một hệ thống web bán game bida có khả năng minh họa các thành phần quan trọng của kiến trúc microservices trong bài toán thương mại điện tử. Cụ thể, báo cáo tập trung vào các mục tiêu sau:

- Tách hệ thống thành nhiều thành phần: frontend, API Gateway, user-service, product-service, order-service, inventory-service, notification-service và các cơ sở dữ liệu riêng.
- Sử dụng API Gateway làm điểm vào thống nhất cho client, che giấu địa chỉ service nội bộ và định tuyến request tới đúng dịch vụ.
- Triển khai luồng dữ liệu storefront/admin mới thông qua SQL Adapter để phục vụ giao diện bán hàng thực tế với SQL Server BidaShopDB.
- Giữ lại luồng microservice gốc sử dụng RabbitMQ nhằm minh họa giao tiếp bất đồng bộ giữa order-service, inventory-service và notification-service.
- Đóng gói toàn bộ môi trường chạy bằng Docker Compose, giúp hệ thống có thể khởi động nhanh và nhất quán trên máy phát triển.
- Xây dựng báo cáo trình bày được cơ sở lý thuyết, thiết kế, triển khai, kết quả minh họa, hạn chế và hướng phát triển.

1.3. Phạm vi

Đề tài tập trung vào quy mô demo phục vụ học tập. Hệ thống được triển khai cục bộ trên máy cá nhân bằng Docker Compose, kết hợp với SQL Server chạy trên host Windows. Phạm vi chức năng bao gồm hiển thị sản phẩm, xem chi tiết, giỏ hàng, đăng ký/dăng nhập, đặt hàng, xem thông báo, đánh giá sản phẩm, blog/thông tin shop và khu vực admin quản lý sản phẩm, người dùng, tồn kho.

Báo cáo không đặt mục tiêu triển khai production trên cloud, chưa tích hợp thanh toán trực tuyến thật, chưa xây dựng đầy đủ hệ thống vận chuyển, chưa có cơ chế scale-out trên Kubernetes và chưa tích hợp đầy đủ giám sát nâng cao như distributed tracing. Các thông tin trong báo cáo được viết theo dữ liệu dự án hiện có và có thêm phần gợi ý chèn ảnh màn hình thực tế để hoàn thiện hồ sơ nộp bài.

Chương 2. CƠ SỞ LÝ THUYẾT VỀ HỆ THỐNG PHÂN TÁN

2.1. Kiến trúc Microservices

Microservices là phong cách kiến trúc trong đó một ứng dụng lớn được chia thành nhiều dịch vụ nhỏ, độc lập về triển khai và chịu trách nhiệm cho một nhóm nghiệp vụ cụ thể. Mỗi service có thể có mã nguồn, cơ sở dữ liệu và vòng đời triển khai riêng. Trong hệ thống web bán gậy bida, các nghiệp vụ như người dùng, sản phẩm, đơn hàng, tồn kho và thông báo được tách thành các service riêng để giảm phụ thuộc trực tiếp giữa các module.

Lợi ích của microservices là khả năng mở rộng từng thành phần theo tải, dễ thay đổi từng nghiệp vụ mà không phải triển khai lại toàn bộ hệ thống, đồng thời tăng khả năng cô lập lỗi. Ví dụ, nếu notification-service gặp lỗi thì luồng hiển thị sản phẩm vẫn có thể tiếp tục hoạt động qua API Gateway và product-service. Tuy nhiên, microservices cũng tạo ra độ phức tạp trong vận hành, vì hệ thống cần quản lý nhiều tiến trình, nhiều cơ sở dữ liệu, giao tiếp qua mạng và đồng bộ dữ liệu không tức thời.

2.2. API Gateway

API Gateway là thành phần đặt ở biên hệ thống, đóng vai trò điểm vào duy nhất cho frontend hoặc các client bên ngoài. Thay vì để trình duyệt gọi trực tiếp vào từng service nội bộ, frontend chỉ gọi tới Gateway. Gateway nhận request, kiểm tra đường dẫn, định tuyến tới service phù hợp hoặc xử lý trực tiếp các route đặc biệt như route /sql/* trong dự án.

Trong hệ thống web bida, API Gateway được xây dựng bằng Node.js/Express. Gateway vừa làm reverse proxy tới các service nội bộ như /users, /products, /orders, /inventory, /notifications, vừa có SQL Adapter để thao tác trực tiếp với SQL Server cho luồng storefront/admin mới. Cách thiết kế này giúp frontend có một endpoint thống nhất, đồng thời tránh publish trực tiếp các port 3001-3005 ra ngoài máy host.

2.3. Giao tiếp REST và Message Broker

REST là kiểu giao tiếp đồng bộ phổ biến trong ứng dụng web. Client gửi request HTTP và chờ response từ server. Các thao tác như lấy danh sách sản phẩm, đăng nhập, xem đơn hàng hoặc cập nhật tồn kho đều có thể biểu diễn bằng các phương thức GET, POST, PUT, PATCH và DELETE. REST đơn giản, dễ kiểm thử bằng trình duyệt, Postman hoặc curl.

Message Broker được sử dụng khi một service cần thông báo sự kiện cho service khác mà không muốn chờ xử lý ngay trong cùng một request. RabbitMQ là message broker của hệ thống. Trong luồng microservice gốc, order-service tạo đơn hàng rồi phát sự kiện order.created. inventory-service nhận sự kiện để kiểm tra tồn kho, sau đó phát kết quả inventory.updated hoặc inventory.failed. notification-service tiếp tục ghi nhận sự kiện để phục vụ thông báo.

2.4. Database-per-service và polyglot persistence

Một nguyên tắc quan trọng của microservices là mỗi service nên sở hữu dữ liệu của chính nó. Cách làm này tránh việc nhiều service cùng truy cập trực tiếp vào một bảng dữ liệu, giảm phụ thuộc dữ liệu và giúp service độc lập hơn. Trong dự án, các service gốc sử dụng PostgreSQL riêng: userdb, productdb, orderdb, inventorydb và notificationdb. Mỗi database được chạy trong một container riêng và lưu dữ liệu qua Docker volume.

Bên cạnh đó, dự án còn sử dụng SQL Server BidaShopDB cho luồng storefront/admin mới. Đây là điểm làm cho hệ thống trở thành mô hình hybrid: vừa giữ kiến trúc microservice gốc để minh họa phân tán, vừa có luồng SQL tập trung để phục vụ giao diện bán hàng có dữ liệu thật. Khi viết báo cáo hoặc trả lời vấn đáp, cần nói rõ hai luồng này để tránh nhầm rằng toàn bộ checkout đều đi qua RabbitMQ.

2.5. Containerization và Docker Compose

Containerization là kỹ thuật đóng gói ứng dụng cùng môi trường chạy, thư viện và cấu hình vào một container image. Với hệ thống nhiều service, container giúp giảm lỗi “chạy được trên máy em nhưng không chạy được trên máy khác”. Docker Compose cho phép mô tả nhiều container trong một file YAML, bao gồm image, build context, biến môi trường, volume, network và port mapping.

Trong dự án, docker-compose.yml định nghĩa các container chính như RabbitMQ, năm PostgreSQL database, năm service Node.js, API Gateway và frontend Nginx. Khi chạy docker compose up --build -d, Docker sẽ build image cho từng service và khởi động toàn bộ stack trong cùng một network nội bộ.

2.6. Observability và xử lý lỗi

Trong hệ thống phân tán, lỗi có thể xảy ra ở nhiều vị trí: service chưa sẵn sàng, database chưa kết nối, RabbitMQ chưa chạy, SQL Server trên host không mở cổng

1433 hoặc Gateway không truy cập được host.docker.internal. Vì vậy hệ thống cần có các endpoint health check và log để người phát triển dễ xác định lỗi.

Dự án cung cấp các lệnh kiểm tra nhanh như curl http://localhost:8081/health và curl http://localhost:8081/sql/health. Bên cạnh đó, RabbitMQ UI tại port 15673 giúp quan sát trạng thái queue và message. Đây là các điểm minh họa cơ bản cho khả năng quan sát hệ thống ở quy mô bài tập lớn.

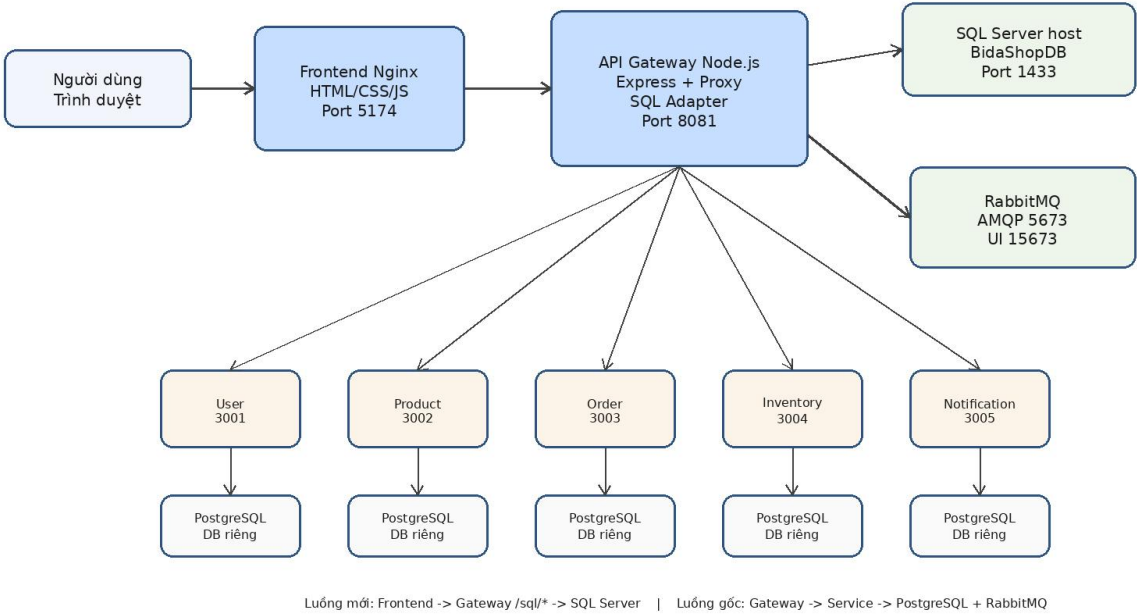
Chương 3. THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc tổng quan

Hệ thống được tổ chức thành bốn lớp chính. Lớp client là frontend đa trang chạy bằng Nginx tại port 5174. Lớp gateway là API Gateway Node.js chạy trong container, publish ra host tại port 8081. Lớp service gồm user-service, product-service, order-service, inventory-service và notification-service. Lớp hạ tầng gồm PostgreSQL cho từng service, RabbitMQ cho message queue và SQL Server trên host cho BidaShopDB.

Kiến trúc tổng quan được mô tả ở hình dưới. Sơ đồ thể hiện rõ hai hướng dữ liệu: hướng /sql/* phục vụ giao diện storefront/admin mới và hướng microservice gốc phục vụ minh họa REST + RabbitMQ.

Hình 1. Kiến trúc tổng quan hệ thống Web bán gậy bida



Hình 1. Kiến trúc tổng quan hệ thống Web bán gậy bida

3.2. Danh sách microservice và trách nhiệm

Các thành phần nghiệp vụ được tách theo trách nhiệm cụ thể. Bảng sau tổng hợp vai trò của từng service trong hệ thống.

Thành phần	Port	Cơ sở dữ liệu/phụ thuộc	Vai trò
------------	------	-------------------------	---------

Frontend Nginx	5174 -> 80	Không có DB riêng	Giao diện khách hàng và admin gồm trang chủ, sản phẩm, giỏ hàng, tài khoản, blog, review, admin dashboard.
API Gateway	8081 -> 8080	SQL Server, các service nội bộ	Điểm vào duy nhất; định tuyến /users, /products, /orders, /inventory, /notifications; xử lý /sql/* qua SQL Adapter.
User Service	3001 nội bộ	user-db PostgreSQL	Quản lý người dùng trong luồng microservice gốc; có JWT_SECRET để hỗ trợ token.
Product Service	3002 nội bộ	product-db PostgreSQL	Quản lý sản phẩm trong luồng microservice gốc.
Order Service	3003 nội bộ	order-db PostgreSQL, RabbitMQ, Product Service	Tạo đơn hàng, phát sự kiện order.created và cập nhật trạng thái đơn sau khi kiểm tra tồn kho.
Inventory Service	3004 nội bộ	inventory-db PostgreSQL, RabbitMQ	Kiểm tra/cập nhật tồn kho, phát kết quả inventory.updated hoặc inventory.failed.
Notification Service	3005 nội bộ	notification-db PostgreSQL, RabbitMQ	Ghi nhận sự kiện và thông báo liên quan tới đơn hàng/tồn kho.
RabbitMQ	5673/15673	AMQP broker	Hàng đợi thông điệp cho luồng event-driven giữa các service.
SQL Server host	1433	BidaShopDB	Nguồn dữ liệu chính cho storefront/admin mới qua route /sql/*.

3.3. Thiết kế API Gateway và SQL Adapter

API Gateway của dự án có hai trách nhiệm lớn. Thứ nhất, Gateway làm proxy cho các service gốc. Các route dạng /users/*, /products/*, /orders/*, /inventory/* và /notifications/* được chuyển tiếp tới service tương ứng trong network Docker. Thứ hai, Gateway trực tiếp xử lý các route /sql/* bằng cách kết nối tới SQL Server qua module sqlServer.js.

SQL Adapter trong Gateway giúp frontend mới đọc/ghi dữ liệu thật từ BidaShopDB. Các nhóm route quan trọng gồm: lấy settings, banner, blog, sản phẩm; đăng ký/đăng nhập; tạo review; xem đơn hàng; checkout; danh sách người dùng; tồn kho; thêm, sửa, xóa sản phẩm trong admin. Nhờ vậy frontend có thể hoạt động như một shop thực tế mà không cần gọi thẳng tới SQL Server từ trình duyệt.

Nhóm API	Endpoint tiêu biểu	Chức năng
----------	--------------------	-----------

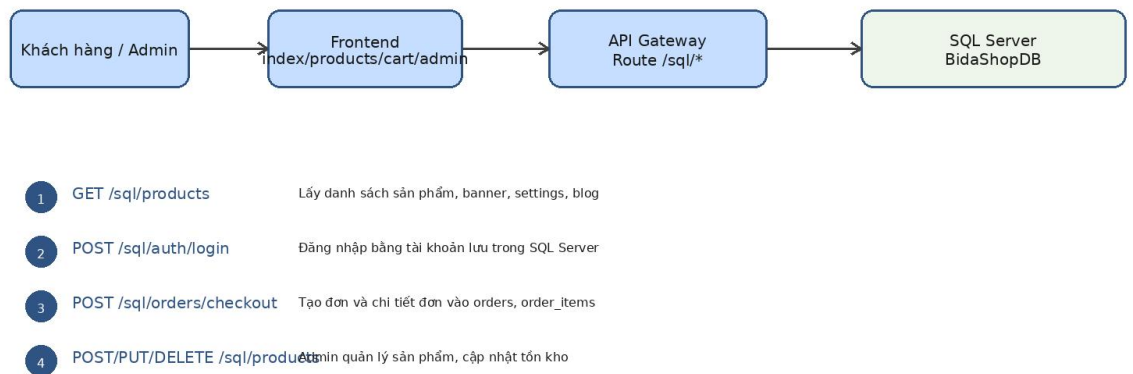
Health	GET /health, GET /sql/health	Kiểm tra Gateway và kết nối SQL Server.
Sản phẩm	GET /sql/products, GET /sql/products/:slugOrId	Lấy danh sách sản phẩm và chi tiết sản phẩm.
Tài khoản	POST /sql/auth/register, POST /sql/auth/login	Đăng ký và đăng nhập user trong SQL Server.
Đơn hàng	GET /sql/orders?userId=..., POST /sql/orders/checkout	Xem đơn hàng và checkout trực tiếp vào bảng orders/order_items.
Đánh giá/Bài viết	GET /sql/reviews, POST /sql/reviews, GET /sql/blog-posts	Hiển thị review, ghi review và đọc bài viết.
Admin	GET /sql/users, GET /sql/inventory, PATCH /sql/inventory/:productId, POST/PUT/DELETE /sql/products	Quản trị người dùng, tồn kho và sản phẩm.
Legacy proxy	ANY /users/*, /products/*, /orders/*, /inventory/*, /notifications/*	Chuyển tiếp tới các microservice gốc.

3.4. Hai luồng dữ liệu trong hệ thống

3.4.1. Luồng storefront/admin mới

Luồng storefront/admin mới là luồng chính để demo giao diện web bán gậy bida. Người dùng thao tác trên frontend, frontend gọi API Gateway theo các endpoint /sql/*, Gateway kết nối SQL Server BidaShopDB và trả dữ liệu cho giao diện. Luồng này phục vụ trải nghiệm mua hàng rõ ràng hơn vì có dữ liệu sản phẩm, ảnh, review, blog, settings và dashboard admin.

Hình 2. Luồng storefront/admin mới qua SQL Adapter



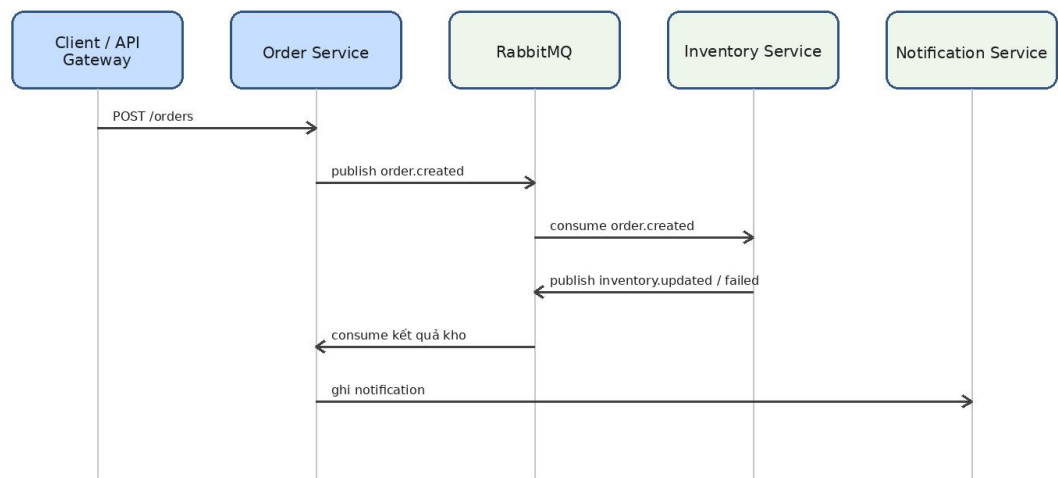
Hình 2. Luồng storefront/admin mới qua SQL Adapter

3.4.2. Luồng microservice gốc qua RabbitMQ

Luồng microservice gốc được giữ lại để thể hiện đúng tinh thần hệ thống phân tán. Khi client gọi POST /orders qua Gateway, order-service tạo đơn hàng ở trạng thái chờ kiểm tra kho, sau đó publish sự kiện order.created vào RabbitMQ. inventory-service nhận sự kiện để kiểm tra tồn kho. Tùy kết quả, inventory-service phát sự kiện cập nhật hoặc thất bại để order-service đổi trạng thái đơn. notification-service ghi log/thông báo sự kiện.

Luồng này không phải luồng checkout chính của frontend mới, nhưng rất quan trọng trong báo cáo vì nó minh họa giao tiếp bất đồng bộ, loose coupling và xử lý nghiệp vụ dạng event-driven.

Hình 3. Luồng đặt hàng microservice gốc qua RabbitMQ



Luồng này minh họa event-driven: Order tạo đơn PENDING, Inventory kiểm tra tồn kho, Notification ghi nhận sự kiện.

Hình 3. Luồng đặt hàng microservice gốc qua RabbitMQ

3.5. Thiết kế dữ liệu

Hệ thống sử dụng hai nhóm dữ liệu. Nhóm thứ nhất là dữ liệu SQL Server trong BidaShopDB, phục vụ giao diện storefront/admin. Nhóm này bao gồm các bảng như products, product_images, product_reviews, users, orders, order_items, notifications, settings, blog_posts và banners. Đây là nguồn dữ liệu được frontend đọc trực tiếp thông qua Gateway.

Nhóm thứ hai là dữ liệu PostgreSQL riêng cho từng microservice gốc. Mỗi service có database riêng và được cấu hình bằng biến môi trường trong docker-compose.yml. Cách này minh họa nguyên lý database-per-service, giúp service không truy cập trực tiếp vào database của nhau.

Nhóm dữ liệu	Bảng/Database	Mục đích
SQL Server BidaShopDB	products, product_images, product_reviews	Quản lý sản phẩm, ảnh và đánh giá phục vụ storefront.
SQL Server BidaShopDB	users	Lưu tài khoản khách hàng/admin cho luồng SQL.
SQL Server BidaShopDB	orders, order_items	Lưu đơn hàng và chi tiết đơn hàng khi checkout qua /sql/orders/checkout.
SQL Server BidaShopDB	notifications, settings, blog_posts, banners	Thông báo, cấu hình shop, bài viết và banner.
PostgreSQL userdb	Bảng người dùng của user-service	Dữ liệu riêng của user-service trong luồng microservice gốc.
PostgreSQL productdb	Bảng sản phẩm của product-service	Dữ liệu riêng của product-service trong luồng microservice gốc.
PostgreSQL orderdb	Bảng đơn hàng của order-service	Dữ liệu riêng của order-service và trạng thái xử lý đơn.
PostgreSQL inventorydb	Bảng tồn kho của inventory-service	Dữ liệu riêng để kiểm tra/cập nhật tồn kho.
PostgreSQL notificationdb	Bảng thông báo của notification-service	Lưu thông báo/sự kiện đã ghi nhận.

3.6. Thiết kế frontend

Frontend của dự án là giao diện đa trang được phục vụ bằng Nginx. Các file chính gồm index.html, products.html, product.html, cart.html, account.html, blog.html, info.html, review.html và admin.html. Tài nguyên giao diện nằm trong thư mục assets với styles.css, store.js, frontend.js, admin.js và thư mục uploads chứa ảnh sản phẩm.

Các chức năng phía khách hàng bao gồm: xem trang chủ, xem danh sách sản phẩm, xem chi tiết sản phẩm, thêm vào giỏ hàng, checkout, đăng ký/đăng nhập, xem đơn hàng, đọc bài viết, xem thông tin shop và đánh giá sản phẩm. Khu vực admin bao

gồm dashboard, quản lý sản phẩm, quản lý tồn kho, xem danh sách user và theo dõi một số thông tin vận hành.

Trang	File/URL	Chức năng chính
Trang chủ	/ hoặc index.html	Hiển thị banner, sản phẩm nổi bật, thông tin giới thiệu shop.
Danh sách sản phẩm	products.html	Liệt kê sản phẩm, lọc/tìm kiếm theo dữ liệu SQL.
Chi tiết sản phẩm	product.html?id=... hoặc slug	Hiển thị thông tin chi tiết, ảnh, giá, tồn kho, review.
Giỏ hàng	cart.html	Lưu sản phẩm đã chọn, tính tổng tiền và gửi checkout.
Tài khoản	account.html	Đăng ký, đăng nhập, xem đơn hàng của người dùng.
Blog/Thông tin/Review	blog.html, info.html, review.html	Hiển thị bài viết, chính sách, đánh giá sản phẩm.
Admin	admin.html	Dashboard quản trị, quản lý sản phẩm, tồn kho, user, đơn hàng.

Chương 4. CÀI ĐẶT VÀ TRIỂN KHAI

4.1. Cấu trúc dự án

Cấu trúc thư mục chính của bản fixed-v2 gồm ba phần quan trọng: api-gateway, frontend và services. api-gateway chứa mã Node.js/Express cho Gateway và SQL Adapter. frontend chứa giao diện tĩnh được Nginx phục vụ. services chứa các microservice gốc, mỗi service có Dockerfile và package.json riêng.

Cấu trúc rút gọn của dự án:

```
billiard-shop-microservices/
├── api-gateway/
│   ├── src/index.js           # Gateway, proxy routes và SQL Adapter routes
│   ├── src/sqlServer.js      # Cấu hình/kết nối SQL Server
│   ├── Dockerfile
│   └── package.json
├── frontend/
│   ├── index.html, products.html, product.html, cart.html
│   ├── account.html, blog.html, info.html, review.html, admin.html
│   └── assets/
│       ├── styles.css
│       ├── store.js
│       ├── frontend.js
│       ├── admin.js
│       └── uploads/
└── services/
    ├── user-service/
    ├── product-service/
    ├── order-service/
    └── inventory-service/
```

```

├── notification-service/
├── docker-compose.yml
├── README.md
└── RUN_GUIDE_WINDOWS.md

```

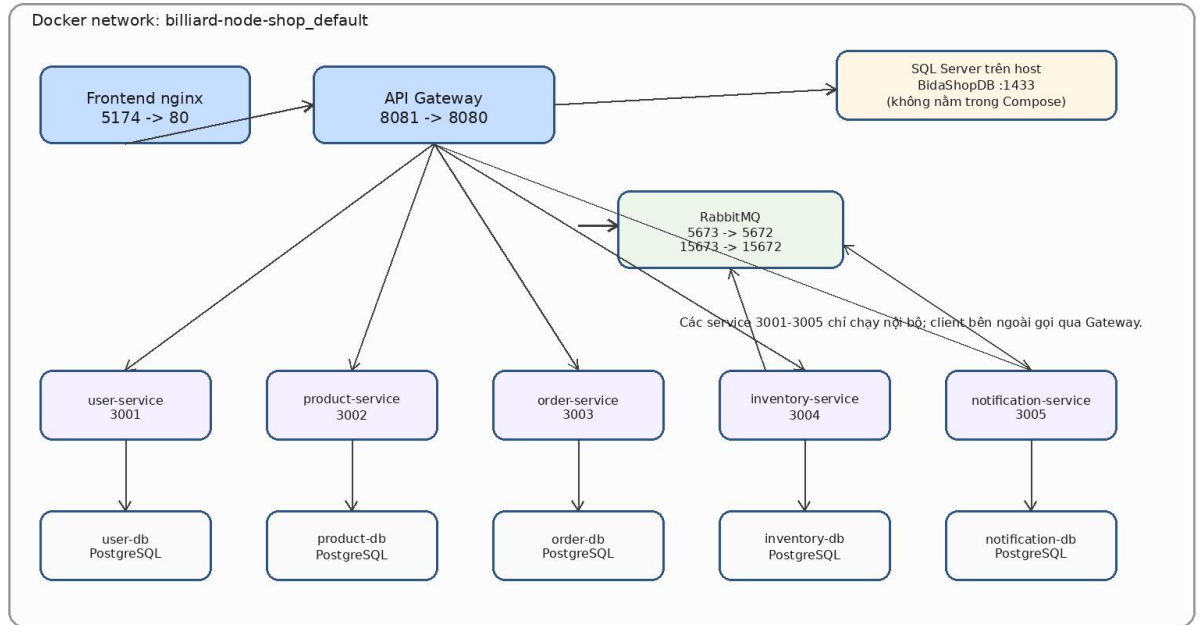
4.2. Công nghệ sử dụng

Lớp	Công nghệ	Vai trò
Frontend	HTML, CSS, JavaScript, Nginx Alpine	Xây dựng giao diện đa trang và phục vụ file tĩnh qua container Nginx.
API Gateway	Node.js, Express, http-proxy-middleware	Định tuyến request, proxy tới service nội bộ và xử lý route /sql/*.
SQL Adapter	mssql package	Kết nối SQL Server host và truy vấn BidaShopDB.
Microservice	Node.js, Express	Xây dựng các service user, product, order, inventory, notification.
Database gốc	PostgreSQL 16 Alpine	Mỗi service gốc có database riêng.
Message Broker	RabbitMQ 3 Management	Truyền sự kiện giữa order-service, inventory-service, notification-service.
Triển khai	Docker Compose	Build và chạy toàn bộ stack trên cùng network nội bộ.
Công cụ kiểm thử	curl, trình duyệt, RabbitMQ UI	Kiểm tra health, API, đơn hàng, thông báo và queue.

4.3. Triển khai bằng Docker Compose

File docker-compose.yml định nghĩa tên stack là billiard-node-shop. Các container PostgreSQL lưu dữ liệu bằng Docker volume. RabbitMQ publish hai port ra máy thật: 5673 cho AMQP và 15673 cho giao diện quản trị. API Gateway publish port 8081, frontend publish port 5174. Các service 3001-3005 không publish ra host để tránh xung đột cổng và đảm bảo client chỉ đi qua Gateway.

Hình 4. Topology triển khai Docker Compose



Hình 4. Topology triển khai Docker Compose

Do Gateway cần kết nối SQL Server đang chạy trên máy Windows host, docker-compose.yml sử dụng `SQLSERVER_HOST=host.docker.internal`, `SQLSERVER_PORT=1433`, `SQLSERVER_DATABASE=BidaShopDB`, `SQLSERVER_USER=sa` và `SQLSERVER_PASSWORD=1234`. Vì vậy, trước khi chạy hệ thống, SQL Server phải đang hoạt động và database BidaShopDB phải có schema/dữ liệu cần thiết.

4.4. Hướng dẫn chạy và kiểm thử

Các bước triển khai trên Windows bằng Docker Desktop:

1. Mở PowerShell tại thư mục `billiard-shop-nodejs-microservices-fixed-v2`.
2. Dọn container cũ nếu đã chạy bản trước: `docker compose down --remove-orphans`.
3. Khởi động toàn bộ hệ thống: `docker compose up --build -d`.
4. Kiểm tra container: `docker ps`.
5. Mở frontend tại `http://localhost:5174`.
6. Mở API Gateway tại `http://localhost:8081`.
7. Mở RabbitMQ UI tại `http://localhost:15673` với tài khoản `guest/guest`.

Một số lệnh kiểm thử nhanh:

```
curl http://localhost:8081/health
curl http://localhost:8081/sql/health
curl http://localhost:8081/sql/products
curl -X PATCH http://localhost:8081/inventory/1 -H "Content-Type: application/json" -d '{"quantity":20}'
curl -X POST http://localhost:8081/orders -H "Content-Type: application/json" -d '{"userId":1,"items":[{"productId":1,"quantity":2}]}'
```

```
curl http://localhost:8081/orders
curl http://localhost:8081/notifications
```

Nếu frontend mở được nhưng không có dữ liệu sản phẩm, cần ưu tiên kiểm tra SQL Server, database BidaShopDB, tài khoản sa, port 1433 và endpoint /sql/health. Nếu luồng microservice gốc không cập nhật thông báo, cần kiểm tra RabbitMQ UI và log của order-service, inventory-service, notification-service.

Chương 5. KẾT QUẢ MINH HỌA

5.1. Kết quả chức năng

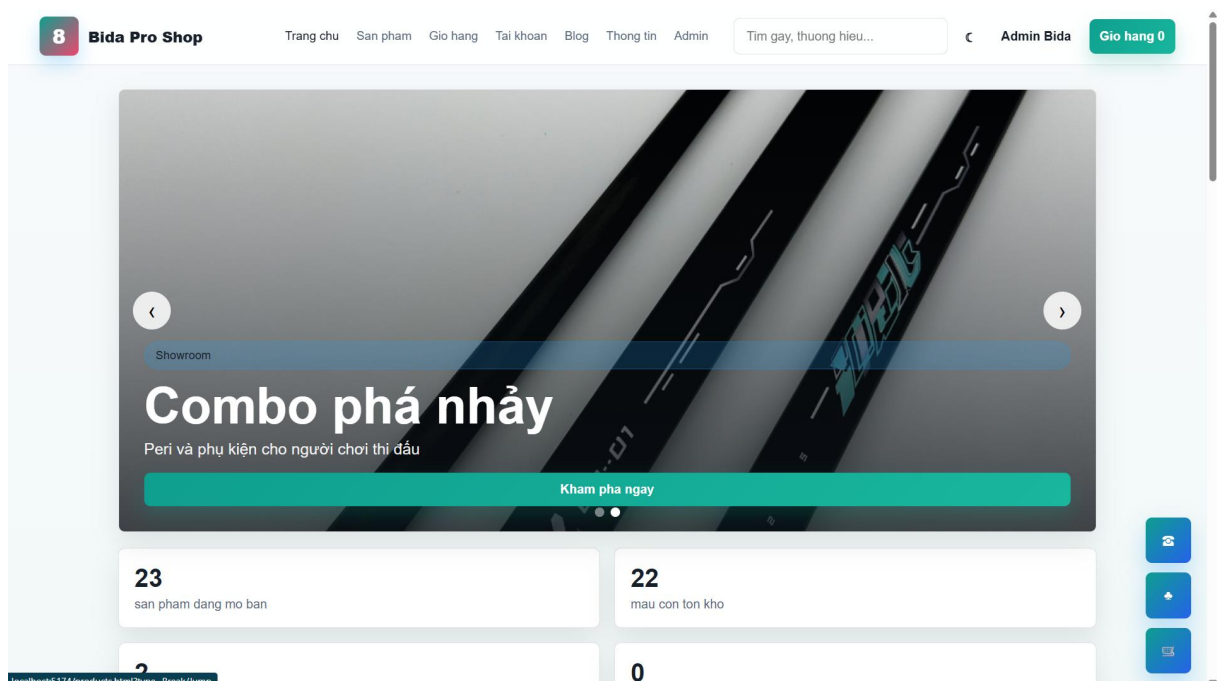
Dựa trên cấu trúc và tài liệu dự án, hệ thống có thể minh họa các nhóm chức năng sau:

Nhóm chức năng	Mô tả	Minh chứng nên chụp
Khởi động hệ thống	Docker Compose build và chạy toàn bộ container.	Ảnh docker ps hoặc Docker Desktop hiển thị container đang chạy.
Frontend khách hàng	Người dùng truy cập trang chủ, danh sách sản phẩm, chi tiết sản phẩm, giỏ hàng.	Ảnh trình duyệt tại localhost:5174, products.html, product.html, cart.html.
Tài khoản	Đăng ký/đăng nhập qua route /sql/auth/register và /sql/auth/login.	Ảnh account.html và response đăng nhập thành công.
Checkout SQL	Frontend tạo đơn qua /sql/orders/checkout, dữ liệu ghi vào orders/order_items.	Ảnh giỏ hàng/checkout và bảng orders trong SQL Server.
Admin	Admin tạo/sửa/xóa sản phẩm, cập nhật tồn kho qua /sql/products và /sql/inventory.	Ảnh admin.html, danh sách sản phẩm, tồn kho trước/sau cập nhật.
Luồng microservice gốc	POST /orders tạo đơn, order-service phát event, inventory-service xử lý, notification-service ghi thông báo.	Ảnh curl POST /orders, RabbitMQ UI, kết quả GET /orders và GET /notifications.
Health check	Gateway và SQL Adapter trả trạng thái OK.	Ảnh terminal chạy curl /health và /sql/health.

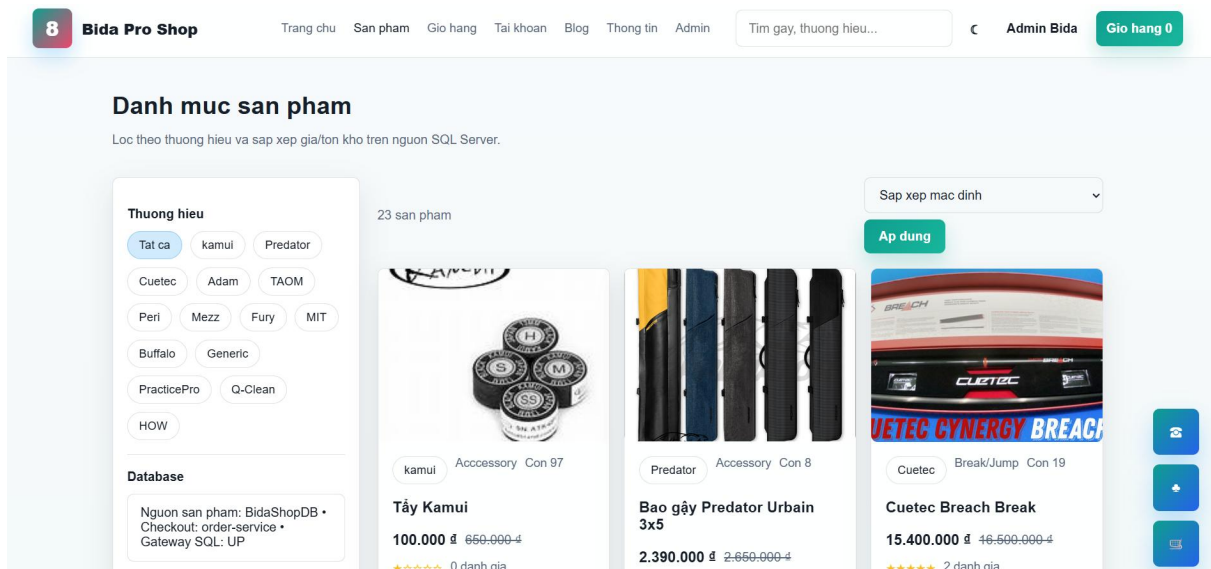
5.2. Demo

```
PS D:\billiard-shop-microservices\billiard-shop-microservices> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED          Status          PORTS
e163272bd761  billiard-node-shop-product-service  "docker-entrypoint.s..." 8 seconds ago   Up 6 seconds    3002/tcp
5755db5886fe  billiard-node-shop-inventory-service "docker-entrypoint.s..." 8 seconds ago   Up 6 seconds    3004/tcp
83aea6efb0e3  billiard-node-shop-user-service     "docker-entrypoint.s..." 8 seconds ago   Up 6 seconds    3001/tcp
7c107ae83f14  billiard-node-shop-notification-service "docker-entrypoint.s..." 8 seconds ago   Up 6 seconds    3005/tcp
a552950cd66a  postgres:16-alpine                 "docker-entrypoint.s..." 8 seconds ago   Up 7 seconds    5432/tcp
7ab13ce522ee  postgres:16-alpine                 "docker-entrypoint.s..." 8 seconds ago   Up 7 seconds    5432/tcp
8b7c7fe9d6a2  postgres:16-alpine                 "docker-entrypoint.s..." 8 seconds ago   Up 7 seconds    5432/tcp
9a2aac1e94fe  postgres:16-alpine                 "docker-entrypoint.s..." 8 seconds ago   Up 7 seconds    5432/tcp
82d4547294d9  postgres:16-alpine                 "docker-entrypoint.s..." 8 seconds ago   Up 7 seconds    5432/tcp
5d1fe9af2c6e  rabbitmq:3-management              "docker-entrypoint.s..." 8 seconds ago   Up 7 seconds    0.0.0.0:5673->5672/tcp, [::]:5673->5672/tcp, 0.0.0.0:15673->15672/tcp, [::]:15673->15672/tcp
PS D:\billiard-shop-microservices\billiard-shop-microservices>
```

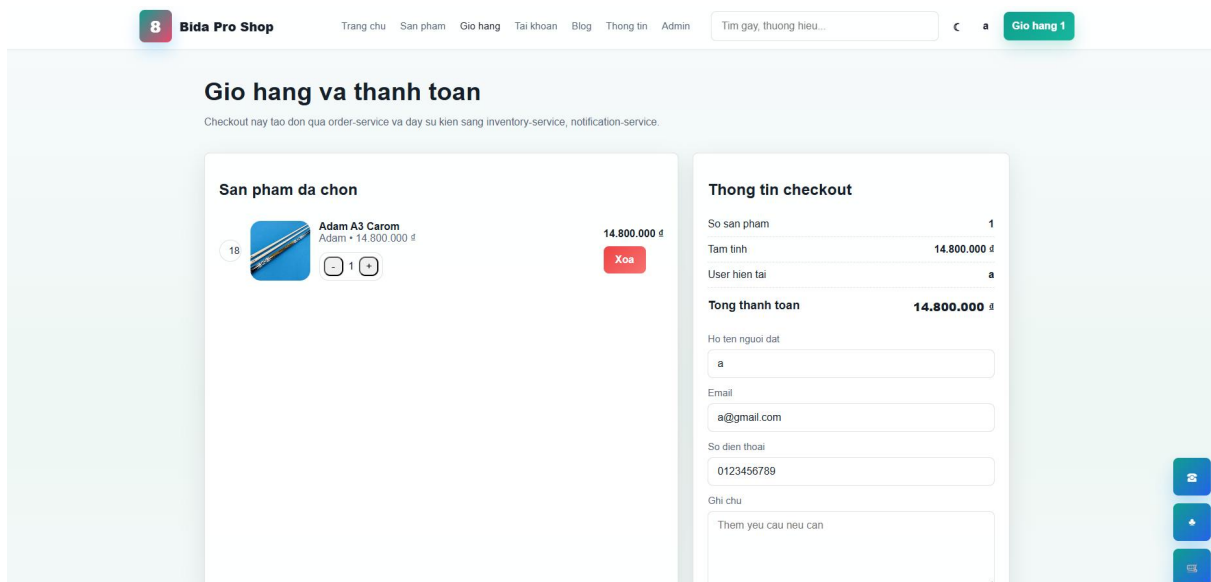
Hình 5. Docker Compose chạy đủ container



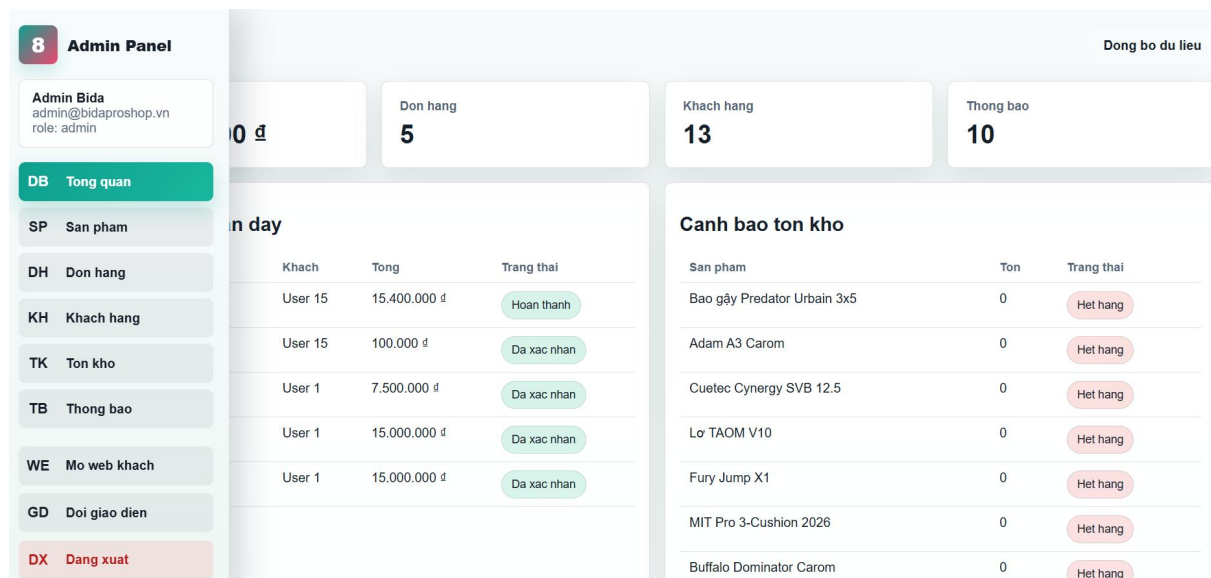
Hình 6. Giao diện trang chủ



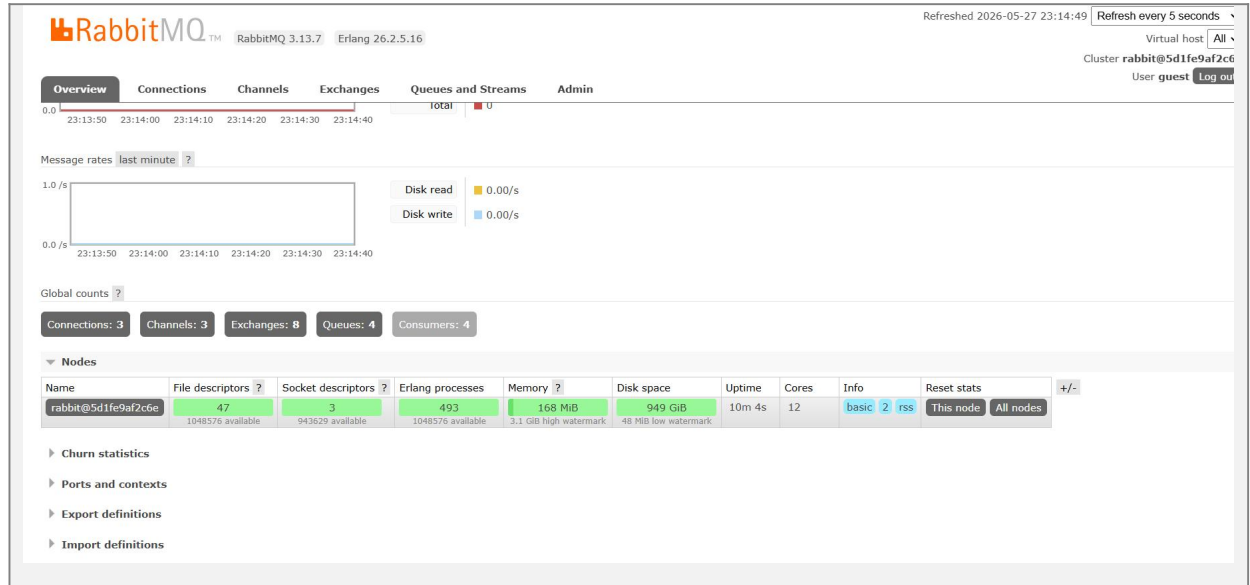
Hình 7. Danh sách/chi tiết sản phẩm



Hình 8. Giỏ hàng và checkout



Hình 9. Khu vực admin



Hình 10. RabbitMQ UI kiểm thử event

5.3. Kịch bản kiểm thử đề xuất

Mã TC	Mục tiêu	Dữ liệu/Thao tác	Kết quả mong đợi
TC01	Kiểm tra Gateway sống	GET /health	Trả về trạng thái OK, chứng tỏ API Gateway hoạt động.
TC02	Kiểm tra SQL Adapter	GET /sql/health	Trả về kết nối SQL Server thành công hoặc thông báo lỗi rõ ràng nếu SQL chưa chạy.
TC03	Lấy danh sách sản phẩm	GET /sql/products	Trả về danh sách sản phẩm từ BidaShopDB.
TC04	Đăng ký tài khoản	POST /sql/auth/register với name, email, password, phone	Tạo user mới trong SQL Server, không trùng email.
TC05	Đăng nhập	POST /sql/auth/login	Trả về thông tin user/token hoặc thông tin đăng nhập hợp lệ.
TC06	Checkout SQL	POST /sql/orders/checkout với userId và items	Tạo bản ghi orders và order_items, giỏ hàng có thể chuyển sang trạng thái đặt thành công.
TC07	Admin thêm sản phẩm	POST /sql/products	Sản phẩm mới xuất hiện trong danh sách sản phẩm.
TC08	Admin cập nhật tồn kho	PATCH /sql/inventory/:productId	Số lượng tồn kho của sản phẩm thay đổi.

TC09	Luồng RabbitMQ	POST /orders với userId và items	Order-service tạo đơn, inventory-service xử lý, notification-service ghi nhận thông báo.
TC10	Xem thông báo	GET /notifications	Trả về danh sách thông báo/sự kiện sau khi tạo đơn.

Chương 6. ĐÁNH GIÁ VÀ KẾT LUẬN

6.1. Những gì đã đạt được

- Hệ thống đã tách được các thành phần chính theo hướng microservices: user, product, order, inventory, notification, API Gateway và frontend.
- API Gateway đóng vai trò điểm vào thống nhất, vừa định tuyến tới các service gốc, vừa cung cấp SQL Adapter cho luồng storefront/admin mới.
- Frontend đa trang mô phỏng tương đối đầy đủ một website bán gậy bida: trang chủ, sản phẩm, chi tiết, giỏ hàng, tài khoản, blog, review và admin.
- Luồng dữ liệu SQL Server giúp giao diện mới sử dụng dữ liệu thực tế trong BidaShopDB, phù hợp để demo web bán hàng.
- Luồng microservice gốc với RabbitMQ minh họa được giao tiếp bất đồng bộ giữa order-service, inventory-service và notification-service.
- Docker Compose giúp đóng gói và chạy toàn bộ môi trường bằng một lệnh, thuận tiện cho demo và kiểm thử trên Windows với Docker Desktop.
- Báo cáo đã phân biệt rõ mô hình hybrid để tránh nhầm lẫn giữa luồng checkout trực tiếp qua SQL Server và luồng đặt hàng event-driven gốc.

6.2. Hạn chế

- Dự án hiện là mô hình hybrid, chưa phải microservices thuần túy vì frontend/storefront mới checkout trực tiếp vào SQL Server qua Gateway thay vì đi qua toàn bộ chuỗi service.
- SQL Server BidaShopDB chạy ngoài Docker Compose nên khi chuyển môi trường cần kiểm tra lại host, port, tài khoản và database.
- Các service gốc sử dụng PostgreSQL riêng nhưng frontend mới ưu tiên dữ liệu SQL Server, vì vậy cần giải thích rõ khi trình bày để tránh mâu thuẫn thiết kế.
- Chưa có cơ chế distributed tracing như OpenTelemetry/Jaeger để theo dõi một request đi qua nhiều service.
- Chưa có rate limiting, circuit breaker hoặc retry đầy đủ ở Gateway cho các request tới service nội bộ.
- Chưa tích hợp thanh toán trực tuyến thật, vận chuyển, phân quyền admin chi tiết và quy trình bảo mật production.
- Chưa có triển khai Kubernetes/multi-node trong bản fixed-v2, hệ thống mới dừng ở Docker Compose cục bộ.

6.3. Hướng phát triển

- Tách rõ hai phiên bản triển khai: phiên bản storefront SQL để demo giao diện và phiên bản microservices thuần để demo phân tán.
- Đưa checkout của frontend mới đi qua order-service và RabbitMQ để đồng nhất với kiến trúc microservices.
- Bổ sung authentication/authorization rõ ràng cho admin, mã hóa mật khẩu và phân quyền theo vai trò.
- Tích hợp OpenTelemetry, Jaeger hoặc Grafana/Prometheus để quan sát log, metric và trace.
- Bổ sung retry, timeout, circuit breaker ở Gateway hoặc trong các service khi gọi phụ thuộc bên ngoài.
- Đóng gói SQL Server hoặc viết script seed database để người khác chạy dự án dễ hơn.
- Triển khai lên cloud/VPS hoặc Kubernetes để minh họa scale-out, rolling update và self-healing.
- Bổ sung test tự động cho API, unit test cho service và test end-to-end cho frontend.

6.4. Kết luận

Đề tài “Phát triển hệ thống Web bán game theo kiến trúc Microservices” đã xây dựng được một mô hình phù hợp để minh họa nhiều khái niệm quan trọng của hệ thống phân tán: tách service theo nghiệp vụ, API Gateway, database-per-service, message broker, giao tiếp REST, event-driven architecture và containerization bằng Docker Compose.

Điểm nổi bật của dự án là có giao diện web bán hàng tương đối đầy đủ, đồng thời vẫn giữ được các service gốc để trình bày bản chất microservices. Tuy còn một số hạn chế do mô hình hybrid và phạm vi demo, hệ thống vẫn đáp ứng tốt mục tiêu học tập, báo cáo và vấn đáp về kiến trúc microservices trong thương mại điện tử. Trong tương lai, hệ thống có thể được phát triển theo hướng microservices thuần hơn, bảo mật tốt hơn, có giám sát tập trung và triển khai trên hạ tầng thực tế.

TÀI LIỆU THAM KHẢO

- [1] Docker Documentation, Docker Compose overview.
- [2] Express.js Documentation, Routing and middleware.
- [3] RabbitMQ Documentation, Tutorials and AMQP concepts.
- [4] PostgreSQL Documentation, relational database system.
- [5] Microsoft SQL Server Documentation, SQL Server connectivity and configuration.
- [6] Nginx Documentation, serving static content and reverse proxy concepts.